



**INSTITUTO
FEDERAL**
Santa Catarina

Listas e Grades

Programação para Aplicativos Móveis

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

tom escuro	#45474B	outer space
acento escuro	#D45113	syracuse red-orange
cor principal	#006674	caribbean current
acento claro	#FFBA08	selective yellow
tom claro	#F5F7F8	anti-flash white

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono

Listas

listas estáticas

■ Column

- posiciona seus filhos (itens) em sequência vertical

exemplos

```
@Composable
fun ListaVertical() {
    Column {
        Text("Ana")
        Text("Bruno")
        Text("Carlos")
    }
}
```

```
@Composable
fun OutraListaVertical() {
    val nomes = listOf("Ana", "Bruno", "Carlos")
    Column {
        nomes.forEach { nome -> Text(nome) }
    }
}
```

Listas

listas estáticas

■ Column

- posiciona seus filhos (itens) em sequência vertical
- não permitem rolagem por padrão
 - `Modifier.verticalScroll` para rolar
- altura suficiente para acomodar os itens
 - `Modifier.height` para fixar altura
 - `Modifier.fillMaxHeight` para preencher a altura disponível
- o posicionamento dos itens é definido com `verticalArrangement`

exemplos

```
@Composable
fun ListaVerticalRolavel() {
    val nomes = listOf("Ana", "Bruno", "Carlos")
    val scrollState = rememberScrollState()
    Column(
        modifier = Modifier.verticalScroll(scrollState)
    ) {
        nomes.forEach { nome -> Text(nome) }
    }
}
```

Listas

listas estáticas

- Column
- Row
 - posiciona seus filhos (itens) em sequência horizontal

exemplos

```
@Composable
fun ListaHorizontal() {
    Row {
        Text("caneta")
        Text("lápis")
        Text("caderno")
    }
}
```

```
@Composable
fun OutraListaHorizontal() {
    val materiais = listOf("caneta", "lápis", "caderno")
    Row {
        materiais.forEach { material -> Text(material) }
    }
}
```

Listas

listas estáticas

■ Column

■ Row

- posiciona seus filhos (itens) em sequência horizontal
- não permitem rolagem por padrão
 - `Modifier.horizontalScroll` para rolar
- largura suficiente para acomodar os itens
 - `Modifier.width` para fixar largura
 - `Modifier.fillMaxWidth` para preencher a largura disponível
- o posicionamento dos itens é definido com `horizontalArrangement`

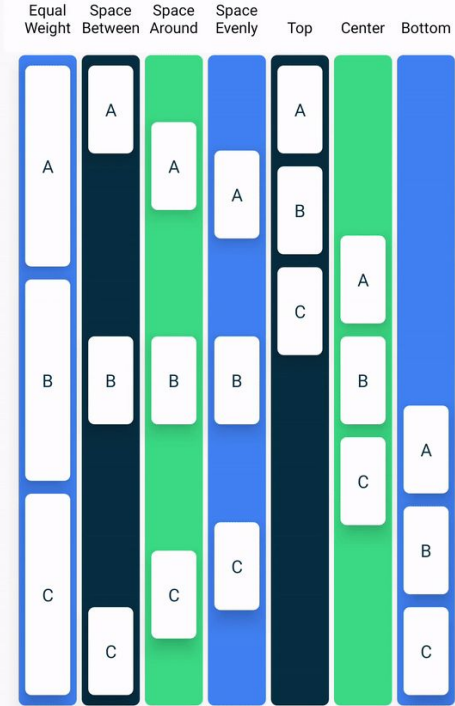
exemplos

```
@Composable
fun ListaHorizontalRolavel() {
    val materiais = listOf("caneta", "lápis", "caderno")
    val scrollState = rememberScrollState()
    Row(
        modifier = Modifier.horizontalScroll(scrollState)
    ) {
        materiais.forEach { material -> Text(material) }
    }
}
```

Listas

posicionando elementos

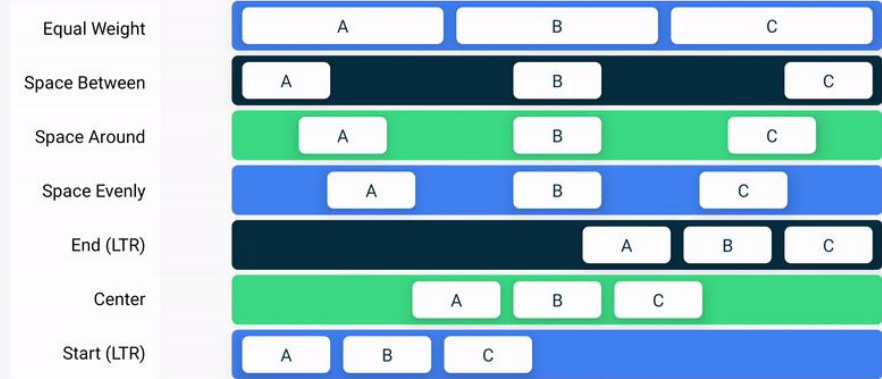
- Arrangement posiciona os itens da lista ao longo do eixo principal
 - **Column**
 - `Arrangement.SpaceBetween`
 - `Arrangement.SpaceAround`
 - `Arrangement.SpaceEvenly`
 - `Arrangement.Top`
 - `Arrangement.Center`
 - `Arrangement.Bottom`



Listas

posicionando elementos

- Arrangement posiciona os itens da lista ao longo do eixo principal
 - Column
 - Row
 - `Arrangement.SpaceBetween`
 - `Arrangement.SpaceAround`
 - `Arrangement.SpaceEvenly`
 - `Arrangement.End`
 - `Arrangement.Center`
 - `Arrangement.Start`



Listas

posicionando elementos

- Arrangement posiciona os itens da lista ao longo do eixo principal
 - Column
 - Row
 - **separando itens**
 - `Arrangement.spacedBy`

exemplos

```
@Composable
fun ListaVerticalRolavel() {
    val nomes = listOf("Ana", "Bruno", "Carlos")
    val scrollState = rememberScrollState()
    Column(
        modifier = Modifier.verticalScroll(scrollState),
        verticalArrangement = Arrangement.spacedBy(14.dp)
    ) {
        nomes.forEach { nome -> Text(nome) }
    }
}
```

Listas

listas preguiçosas (lentas)

■ LazyColumn/LazyRow

- posiciona em sequência apenas os itens visíveis
- melhora o desempenho em listas grandes
- contêiner rolável
- funções internas
 - `item` adiciona apenas um item à lista
 - `items` adiciona vários itens à lista

exemplos

```
@Composable
fun ListaVerticalPreguicosa() {
    val nomes = listOf("Ana", "Bruno", "Carlos")
    LazyColumn {
        item {
            Text("Alunos")
        }
        items(nomes) { nome ->
            Text(nome)
        }
        item {
            Text("Fim da lista")
        }
    }
}
```

Listas

listas preguiçosas (lentas)

■ LazyColumn/LazyRow

- posiciona em sequência apenas os itens visíveis
- melhora o desempenho em listas grandes
- contêiner rolável
- funções internas
 - item adiciona apenas um item à lista
 - itens adiciona vários itens à lista

exemplos

```
data class Aluno(  
    val id: Int,  
    val nome: String,  
    val curso: String  
)  
  
@Composable  
fun ListaVerticalPreguicosa() {  
    val alunos = listOf(  
        Aluno(1, "Ana", "ADS"),  
        Aluno(2, "Bruno", "Engenharia de Telecomunicações"),  
        Aluno(3, "Carlos", "ADS")  
    )  
    LazyColumn {  
        items(  
            items = alunos,  
            key = { aluno -> aluno.id }  
        ) { aluno -> Text(aluno.nome) }  
    }  
}
```

Listas

listas preguiçosas (lentas)

■ LazyColumn/LazyRow

- posiciona em sequência apenas os itens visíveis
- melhora o desempenho em listas grandes
- contêiner rolável
- funções internas
 - `item` adiciona apenas um item à lista
 - `items` adiciona vários itens à lista
 - `itemsIndexed` informa posição do item

exemplos

```
data class Aluno(  
    val id: Int,  
    val nome: String,  
    val curso: String  
)  
  
@Composable  
fun ListaVerticalPreguicosa() {  
    val alunos = listOf(  
        Aluno(1, "Ana", "ADS"),  
        Aluno(2, "Bruno", "Engenharia de Telecomunicações"),  
        Aluno(3, "Carlos", "ADS")  
    )  
    LazyColumn() {  
        itemsIndexed(alunos) { indice, aluno ->  
            Text(text = "${indice + 1}. ${aluno.nome}")  
        }  
    }  
}
```

Grades

grades preguiçosas (lentas)

■ LazyColumnGrid/LazyRowGrid

- posiciona em mais de uma coluna/linha os itens visíveis
- melhora o desempenho em conjuntos grandes
- contêiner rolável
- o parâmetro columns/rows define como as células são formadas
 - GridCells.Fixed número de colunas/linhas fixo

exemplos

```
@Composable
fun ListaVerticalPreguicosa() {
    val alunos = listOf(
        Aluno(1, "Ana", "ADS"),
        Aluno(2, "Bruno", "Engenharia de Telecomunicações"),
        Aluno(3, "Carlos", "ADS")
    )
    LazyVerticalGrid(
        columns = GridCells.Fixed(2)
    ) {
        items(alunos) { aluno ->
            AlunoItem(aluno.nome)
        }
    }
}
```

Grades

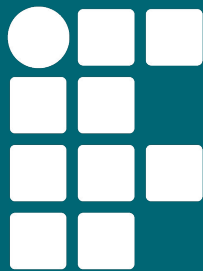
grades preguiçosas (lentas)

■ LazyColumnGrid/LazyRowGrid

- posiciona em mais de uma coluna/linha os itens visíveis
- melhora o desempenho em conjuntos grandes
- contêiner rolável
- o parâmetro columns/rows define como as células são formadas
 - GridCells.Fixed número de colunas/linhas fixo
 - GridCells.Adaptive largura/altura mínima da coluna/linha

exemplos

```
@Composable
fun ListaVerticalPreguicosa() {
    val alunos = listOf(
        Aluno(1, "Ana", "ADS"),
        Aluno(2, "Bruno", "Engenharia de Telecomunicações"),
        Aluno(3, "Carlos", "ADS")
    )
    LazyVerticalGrid(
        columns = GridCells.Adaptive(minSize = 128.dp)
    ) {
        items(alunos) { aluno ->
            AlunoItem(aluno.nome)
        }
    }
}
```



**INSTITUTO
FEDERAL**
Santa Catarina

Listas e Grades

Programação para Aplicativos Móveis

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

